A Two-Stage YOLOv8 and MobileNetV2 Framework for Automated Motorcycle Helmet Violation Detection

Lavnish Kumar Singh
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
lavnishdod@gmail.com

Dr. Puneet Kumar
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India

***Abstract*—Motorcycle riders remain among the most vulnerable road users, and a large share of the injuries recorded in motorcycle accidents involve riders who were not wearing a helmet at the time of the crash. Enforcement of helmet laws is still carried out almost entirely by traffic police stationed at fixed points, which does not scale well to the volume of two-wheeler traffic in most cities. This paper presents a two-stage deep learning pipeline that automates helmet-violation detection directly from ordinary traffic video. In the first stage, a YOLOv8n detector pretrained on the COCO dataset locates motorcycles and pedestrians in each frame and associates nearby persons with a detected motorcycle to isolate likely riders, without any additional training. In the second stage, the head region of each rider is cropped and passed to a MobileNetV2 classifier, fine-tuned through transfer learning, which decides whether the rider is wearing a helmet. Every negative decision is time-stamped, logged to a CSV file, and saved as an image snapshot, producing an auditable violation record rather than a simple on-screen alert. The classifier was trained and evaluated on a fixed, reproducible 70/15/15 split of a public helmet dataset (1,666 head-crop images across two classes), and on a held-out test set of 252 images that was never used for training or model selection it reached an overall accuracy of 90.5%, with a macro-averaged precision, recall and F1-score of 0.905. We discuss the design choices behind separating detection from classification, report the confusion matrix and per-class metrics, and outline the limitations of the current system along with directions for future work such as multi-object tracking and automatic license-plate association.**

***Index Terms*—helmet detection, YOLOv8, convolutional neural network, transfer learning, road safety, object detection, MobileNetV2**

I. Introduction

Two-wheelers make up a disproportionately large share of vehicles on the road in many developing countries, and motorcycle riders are consistently reported as one of the road-user groups most exposed to fatal head trauma in a crash. A properly worn helmet is one of the few protective measures available to a rider, and its effect on reducing the severity of head injury has been studied extensively in the road-safety literature [17]. Despite this, helmet non-compliance is still common, particularly for short trips within a city, and the responsibility for catching violators currently rests almost entirely on traffic police who are physically present at an intersection or checkpoint. This does not scale: a single officer can only watch so many riders at once, and enforcement effectively stops the moment nobody is looking.

The obvious response is to move some of that watching onto a camera and a model instead of a person. Video surveillance infrastructure is already installed at most major intersections for other purposes, and modern object-detection networks are fast enough to process that video in real time on comparatively modest hardware. What is missing in a lot of existing city-level CCTV systems is not the camera, but the software layer that turns a video feed into an actionable violation record automatically, at scale, and without requiring a human to keep staring at a monitor.

Over the last several years, the object-detection literature has moved from two-stage, region-proposal-based detectors toward single-stage detectors in the YOLO family, which trade a small amount of localization precision for a large gain in inference speed [18]. Because of this, YOLO and its variants have become the default building block for most recent helmet-detection systems, and the published work in this space can broadly be split into two design patterns. The first treats helmet detection as a single object-detection problem with classes such as “with helmet” and “without helmet” detected directly by one network [1], [5]–[7]. The second splits the problem into two stages: first find the motorcycle and the rider, then decide separately whether that rider has a helmet on, an approach used by Jia et al. [8] with two successive YOLOv5 passes. Both patterns appear repeatedly in the literature, and each has real trade-offs, which we discuss further in Section II.

This paper follows the second pattern, but combines it with a lighter and more common architectural pairing: a general-purpose, COCO-pretrained YOLOv8n detector for locating riders, and a dedicated, separately trained MobileNetV2 classifier for the actual helmet decision. The reasoning behind this split, rather than training one large detector on custom helmet classes, is discussed in Section III. Our specific contributions in this paper are as follows:

- We describe a two-stage pipeline in which the rider-localization stage requires no training at all, since COCO

already contains `person` and `motorcycle` classes, and only the helmet classifier needs a custom, task-specific dataset.
- We evaluate the classifier on a fixed, seed-controlled 70/15/15 train/validation/test split rather than a single random holdout, so that the reported test accuracy is reproducible and was measured exactly once on data the model never saw during training or checkpoint selection.
- We report full precision, recall, F1-score and confusion-matrix results on the held-out test set, rather than training accuracy alone, and we discuss where the model's errors concentrate.
- We implement violation logging as a first-class part of the system, recording a timestamp, confidence score and image snapshot for every flagged rider, so the output is closer to something a traffic authority could actually act on.

The rest of the paper is organized as follows. Section II reviews related work on helmet detection using classical and deep-learning methods. Section III describes the proposed system architecture, dataset and training procedure. Section IV presents the experimental results. Section V concludes the paper and discusses directions for future work.

II. Related Work

Early attempts at automatic helmet detection relied on handcrafted visual features rather than learned ones. Silva et al. used background subtraction with an adaptive mixture of Gaussians together with hybrid descriptors to separate a rider's head region from the rest of the frame, and Chiu et al. combined a vertical projection of binarized images with rider counting to estimate helmet usage in a video stream [9]. These methods worked reasonably well under the specific lighting and camera-angle conditions they were tuned for, but their accuracy dropped quickly once the scene changed, and they generally could not be retrained on new footage without redesigning the feature-extraction pipeline by hand.

The shift toward convolutional neural networks changed this. Dasgupta et al. proposed one of the earlier CNN-based systems capable of handling multiple motorcycle riders in a single frame rather than assuming one rider per detection [15], and Rohith et al. built a deep-learning pipeline specifically aimed at motor-vehicle-department style enforcement footage [16]. Around the same period, Siebert and Lin trained a RetinaNet-based detector on self-collected data and reported that helmet-use classification, while promising, was still noticeably harder in cluttered street scenes than in curated benchmark datasets [17], which is a limitation that later YOLO-based systems have also run into.

Once the YOLO family matured past YOLOv3, most subsequent helmet-detection papers adopted it as the backbone detector, largely because of its speed advantage over two-stage detectors such as Faster R-CNN. A number of papers train a YOLO variant directly on a three- or four-class problem, typically *helmet*, *no-helmet*, and sometimes *number plate* or *rider*, and detect all of them in a single forward pass [1], [5]–[7], [14]. This single-stage design is attractive because it is simple to deploy, but it also means the network has to learn to localize a comparatively small object, the helmet or head, inside a much larger and more cluttered traffic scene, which several of these papers note as a source of missed detections at a distance or under occlusion.

The alternative, two-stage design first isolates the rider and only then makes the helmet decision on that cropped region. Jia et al. implemented this using two successive passes of an improved YOLOv5 detector, one for the motorcycle and rider, and a second for the helmet itself, and reported that this ordering improved robustness in complex urban traffic compared to detecting the helmet directly in the full frame [8]. Aboah et al. approached the same underlying problem from the angle of the 2023 AI City Challenge, combining YOLOv5 detections with an ensemble of classifiers and reported that ensembling gave a measurable improvement over any single classifier on their benchmark [2], while a separate AI City Challenge submission focused specifically on multi-class helmet-violation detection under the competition's constrained evaluation protocol [9].

More recent work has started pairing helmet detection with an additional downstream task, most commonly automatic license-plate recognition, so that a flagged violation is immediately linked to an identifiable vehicle rather than just an image. Systems built on the NVIDIA TAO toolkit and YOLOv8 have been reported for Indian smart-city traffic scenarios with this exact goal [13], and a Springer-published pipeline combines YOLOv8-based multi-class violation detection with an OCR stage for plate recognition [10]. Other papers have pushed in the direction of edge deployment instead, running YOLOv9 or YOLOv8 on Raspberry Pi hardware, sometimes with a dedicated AI accelerator, to bring the cost of a deployable unit down to something a municipality could realistically install at scale [3], [4]. A broader review by Terven et al. traces the architectural evolution from YOLOv1 through YOLOv8 and YOLO-NAS and is a useful reference for why later YOLO versions were adopted so quickly across this literature [18].

It is also worth separating these systems by where they are meant to run, since that choice shapes the rest of the design as much as the choice of detector does. Server- or cloud-based systems, which describes most of the work discussed so far, can afford a larger backbone and a heavier ensemble because inference happens on hardware with a GPU and no strict power budget. Edge-deployed systems built for a Raspberry Pi or a similar single-board computer, on the other hand, have to work within a much tighter compute and power envelope, which is why those papers tend to favor smaller YOLO variants and dedicated accelerators over raw accuracy [3], [4]. Our own choice of a nano-sized YOLOv8 detector paired with MobileNetV2, discussed in Section III, sits closer to the edge-friendly end of this spectrum, even though the current implementation was evaluated on general-purpose CPU hardware rather than a dedicated embedded board.

Across this body of work, a recurring gap is methodological rather than architectural: many papers report a single training-

time or validation accuracy figure without describing whether that figure came from data the model never saw during training, and few report a full confusion matrix or per-class precision and recall alongside overall accuracy. This makes it difficult to compare results across papers, since a model can post a high overall accuracy while still performing poorly on the minority class. In the present work we deliberately separate training, validation and test data with a fixed random seed, evaluate the test set exactly once, and report full per-class metrics rather than accuracy alone, specifically to address this gap in a small way.

III. Proposed Methodology

A. System Overview

The proposed system takes an ordinary video file or camera stream as input and processes it frame by frame. Fig. 1 shows the overall flow. Each frame first passes through Stage 1, where a YOLOv8n detector locates motorcycles and persons and associates nearby persons with a detected motorcycle to identify likely riders. Each rider's head region is then cropped using a simple positional heuristic and passed to Stage 2, a MobileNetV2 classifier that outputs a helmet or no-helmet decision with an associated confidence score. Frames are annotated with the result, and every no-helmet decision is written to a violation log together with a timestamp, the frame number, the confidence score, and a saved image snapshot of the cropped region.

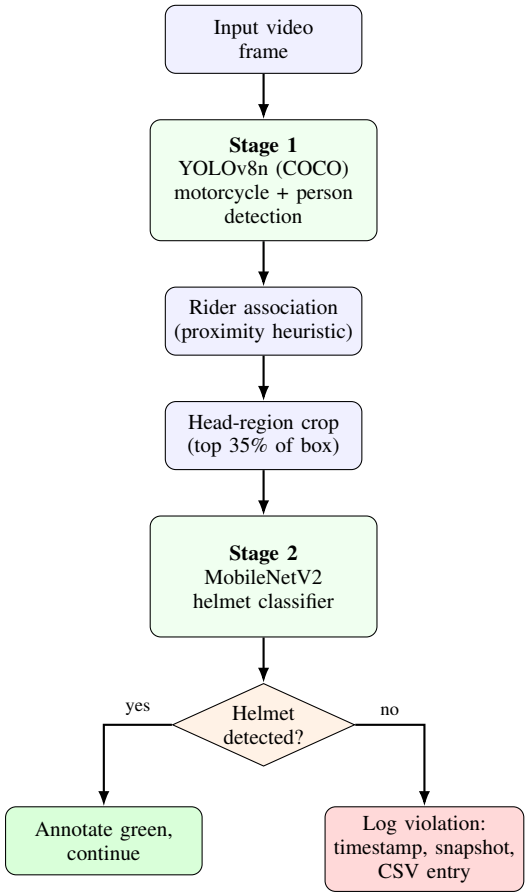


Fig. 1. Overall system architecture. Stage 1 requires no training since COCO already provides `person` and `motorcycle` classes; only Stage 2 is trained on task-specific data.

B. Stage 1: Rider Detection

The first stage uses YOLOv8n with its publicly released weights, trained on the COCO dataset, without any fine-tuning. This is a deliberate choice rather than an oversight: COCO already contains both `person` and `motorcycle` as annotated classes, so there is little to be gained from collecting and labeling a custom dataset just to re-derive a detector that COCO's own pretrained weights already provide. The "n" (nano) variant of YOLOv8 was selected specifically for its inference speed, since the target use case is continuous video rather than single-image analysis.

A person box is only treated as a candidate rider if it falls within a proximity zone around a detected motorcycle box, computed by widening the motorcycle's bounding box horizontally and checking whether the person's box center falls inside the widened region. This step exists because raw person detections in a traffic scene include pedestrians and bystanders who have nothing to do with the motorcycle, and passing every person in the frame to the classifier would both waste computation and introduce irrelevant false positives into the violation log.

C. Stage 2: Helmet Classification

Rather than training a single detector to localize the helmet directly inside the full traffic frame, the second stage works on the cropped head region alone, treating helmet detection as a binary image-classification problem. The head region is approximated as the top 35% of a rider's bounding box, a heuristic that avoids the added cost of a pose-estimation model while still being reliable across the range of postures a rider takes on a moving motorcycle.

The classifier itself is a MobileNetV2 backbone, pretrained on ImageNet and fine-tuned end to end on the helmet dataset described below, with its final classification layer replaced to output two classes. MobileNetV2 was chosen over a heavier backbone because it is small enough to run in real time alongside the detection stage on ordinary hardware, and because transfer learning from ImageNet substantially reduces the amount of task-specific labeled data needed to reach a usable accuracy, which matters given that helmet/no-helmet head crops are a comparatively small and specialized dataset. Input images are resized to 128×128 pixels and normalized using standard ImageNet statistics before being passed to the network.

Separating detection and classification into two stages, rather than training one YOLO model on *helmet* and *no-helmet* classes directly, was also a deliberate design decision. Classifying an already-cropped, roughly centered head image is a substantially easier learning problem than localizing a small helmet region inside a cluttered full frame, and it requires far less labeled data to reach a reasonable accuracy. It

TABLE I
DATASET SPLIT SUMMARY (FIXED SEED = 42)

Class	Train	Validation	Test
With Helmet	599	128	130
Without Helmet	566	121	122
Total	1,165	249	252

also keeps the two stages independent: the classifier can later be retrained, swapped for a different backbone, or extended with more classes without touching the detector, and vice versa.

D. Dataset and Preprocessing

The classifier was trained on head-region crops derived from a publicly available helmet-detection dataset annotated in Pascal VOC format, with bounding boxes labeled *With Helmet* and *Without Helmet* in addition to other object classes that were not used, such as license plates. Each labeled box was cropped out of its source image with a small fixed padding of 6 pixels on every side, producing 857 *with_helmet* images and 809 *without_helmet* images, for a total of 1,666 labeled crops. Standard data augmentation, consisting of random horizontal flips and mild brightness/contrast jitter, was applied during training to reduce overfitting on this relatively modest dataset size.

To obtain a result that can be reported and reproduced reliably, the dataset was split into training, validation and test subsets in a 70/15/15 ratio using a fixed random seed, with the exact file assignment for each split recorded in a manifest file. The model was trained using only the training split, model selection across epochs was performed using the validation split, and the test split was set aside completely and evaluated only once, after training was finished, to produce the final results reported in Section IV. Table I summarizes the resulting split.

E. Training Configuration

The classifier was trained for 15 epochs using the Adam optimizer with a learning rate of 1×10^{-4} and a batch size of 16, using cross-entropy loss. The checkpoint with the highest validation accuracy across all epochs was retained as the final model, which reached a best validation accuracy of 94.8% on the held-out validation split. Table II lists the full training configuration for reference.

Algorithm 1 summarizes the full inference-time procedure applied to each incoming video frame, combining both stages described above into a single pass.

F. Violation Logging

Rather than simply drawing a bounding box on screen, every frame in which a rider is classified as not wearing a helmet triggers a log entry containing the frame number, a timestamp, the classifier's confidence score, and a cropped snapshot image of the violating rider, all written to a CSV file. This was included specifically to move the system output closer to something usable for a traffic-enforcement workflow rather than a purely visual demonstration, though as discussed in Section V, the current implementation logs a violation on every frame a rider is visible rather than once per unique rider, which is one of the clearer directions for future improvement.

TABLE II
TRAINING HYPERPARAMETERS

Hyperparameter	Value
Backbone	MobileNetV2 (ImageNet-pretrained)
Input resolution	128×128
Optimizer	Adam
Learning rate	1×10^{-4}
Batch size	16
Epochs	15
Loss function	Cross-entropy
Augmentation	Horizontal flip, color jitter
Framework	PyTorch / torchvision

Algorithm 1 Per-frame helmet violation detection

Require: Video frame F, YOLOv8n detector D, classifier C
$M \leftarrow D(F, \text{class} = \texttt{motorcycle})$ ▷ motorcycle boxes
$P \leftarrow D(F, \text{class} = \texttt{person})$ ▷ person boxes
$R \leftarrow \emptyset$ ▷ rider boxes
for each $p \in P$ **do**
  if $\exists m \in M$ such that p is near m **then**
    $R \leftarrow R \cup \{p\}$
  end if
end for
for each $r \in R$ **do**
  $h \leftarrow$ top 35% crop of r
  $(\text{label}, \text{conf}) \leftarrow C(h)$
  if label $=$ `no_helmet` **then**
    log(timestamp, frame_id, conf, snapshot(h))
    draw red box on r
  else
    draw green box on r
  end if
end for

IV. EXPERIMENTAL RESULTS

A. Test Set Evaluation

The final classifier was evaluated exactly once on the 252-image held-out test set (130 *with_helmet*, 122 *without_helmet*), which was not used at any point during training or checkpoint selection. Table III reports precision, recall and F1-score for both classes together with the macro-averaged values.

The overall test accuracy was 90.5% (228 correct out of 252 images). Table IV shows the full confusion matrix. Of the 130 helmeted riders in the test set, 115 were correctly classified and 15 were misclassified as not wearing a helmet; of the 122 riders without a helmet, 113 were correctly classified and 9 were misclassified as wearing one.

The two classes perform comparably, with neither one dominating at the expense of the other, which suggests the model has not simply learned to favor the majority class in

TABLE III
PER-CLASS TEST SET METRICS

Class	Precision	Recall	F1-score	Support
With Helmet	0.927	0.885	0.906	130
Without Helmet	0.883	0.926	0.904	122
Macro avg.	0.905	0.905	0.905	252

TABLE IV
CONFUSION MATRIX ON THE HELD-OUT TEST SET

Actual	Predicted	
	With Helmet	Without Helmet
With Helmet	115	15
Without Helmet	9	113

the training data. The slightly higher false-negative count for the *with_helmet* class (15 helmeted riders misclassified as unhelmeted) compared to the false-positive count for *without_helmet* (9 unhelmeted riders misclassified as helmeted) is a pattern also reported informally in related work on cluttered street scenes [17], and is consistent with cases where partial occlusion, unusual head angles, or helmets that blend visually with hair or clothing color make the correct class harder to separate from a small, cropped image alone.

B. Pipeline-Level Observations

Beyond the classifier's own test-set metrics, the complete two-stage pipeline, including the untrained YOLOv8n detection stage, the rider-association heuristic, and the trained classifier, was run end to end on unseen traffic video to confirm the full system operates correctly outside of isolated component testing. On standard CPU hardware without GPU acceleration, the pipeline processed video at approximately 9 to 11 frames per second, which is sufficient for offline or near-real-time analysis of recorded footage, though a GPU or a more aggressive frame-skipping strategy would likely be needed for strict real-time deployment on live camera feeds at higher resolutions.

From a deployment standpoint, the two stages have fairly different resource profiles, which is a useful property when deciding where to run each one. Stage 1, since it reuses an off-the-shelf pretrained detector, needs no retraining if deployed at a new intersection with a different camera angle, though its accuracy on that new viewpoint would still depend on how well COCO's own training distribution covers similar traffic scenes. Stage 2, by contrast, is the part that would most likely need to be retrained or fine-tuned again for a genuinely different environment, since head-crop appearance varies more with local helmet styles, clothing and lighting than a generic person or motorcycle detection does. This suggests that the more expensive part of adapting the system to a new city or region is collecting fresh helmet-classification data, not retraining the detector.

C. Comparison with Reported Results in Related Work

Direct numerical comparison with the systems reviewed in Section II is difficult because most of them do not evaluate on a common benchmark, and several report only a single accuracy figure without a corresponding precision/recall breakdown or a confirmation that the reported number came from a genuinely held-out test set. With that caveat, the overall accuracy obtained here, 90.5% on 252 held-out test images, sits in a broadly similar range to figures reported for other CNN- and YOLO-based helmet classifiers trained on comparably sized custom datasets, while remaining below the highest accuracy figures reported by systems trained on substantially larger or more curated datasets [13]. Given that the dataset used here was a single public source of under two thousand labeled crops, this level of accuracy is a reasonable starting point rather than a finished result, and the discussion in Section V outlines the most direct routes to closing that gap.

V. CONCLUSION AND FUTURE WORK

This paper presented a two-stage pipeline for automated motorcycle helmet-violation detection, combining a pretrained, untrained-by-us YOLOv8n detector for rider localization with a MobileNetV2 classifier fine-tuned specifically for the helmet decision. Separating the two stages kept the harder, task-specific learning problem, helmet classification, confined to a comparatively simple, data-efficient binary classifier operating on cropped head regions, rather than asking a single detector to localize a small helmet region inside a full, cluttered traffic scene. Evaluated on a fixed and reproducible held-out test split that was touched exactly once, the classifier reached 90.5% accuracy with balanced precision and recall across both classes, and the complete pipeline, including violation logging with timestamps and image snapshots, was verified to run correctly end to end on real traffic footage.

The current system has clear limitations that suggest concrete next steps. Violations are currently logged on a per-frame basis rather than a per-rider basis, so a single unhelmeted rider visible for several seconds generates many log entries rather than one; incorporating a lightweight multi-object tracker, such as ByteTrack, would let the system count and log each violation once per rider rather than once per frame. The training dataset, while sufficient to reach a reasonable accuracy, was drawn from a single public source and a larger, more geographically and visually diverse dataset would likely improve robustness to the occlusion and lighting conditions that appear to drive most of the current misclassifications. Finally, linking a detected violation to an automatically read license plate, an approach already explored in related work [10], [13], would move the system from a detection demonstration to something closer to a deployable enforcement tool, and is a natural direction for extending this work.

REFERENCES

[1] C. Vishnu *et al.*, "Helmet violation detection using YOLO v2 deep learning framework," in *Proc. IEEE Conf.*, 2022.

[2] A. Aboah *et al.*, "Real-time helmet violation detection using YOLOv5 and ensemble learning," *arXiv preprint arXiv:2304.09246*, 2023.

[3] "YOLO-based helmet detection and intelligent parking system: A case study," *Int. J. Sustainable Development and Planning (IIETA)*, vol. 15, no. 5, pp. 1075–1088, 2025.
[4] "Edge-based AI solution for enhancing urban safety: helmet compliance monitoring with YOLOv9 on Raspberry Pi," *Discover Internet of Things*, Springer, 2025.
[5] "Helmet detection based on improved YOLO v8," *RJPN J. Emerging Technologies and Novel Research*, 2024.
[6] "Helmet detection system using YOLOv5," *Int. J. Novel Research and Development (IJNRD)*, 2023.
[7] "Optimizing helmet detection with hybrid YOLO pipelines: A detailed analysis," *arXiv preprint arXiv:2412.19467*, 2024.
[8] W. Jia *et al.*, "Real-time automatic helmet detection of motorcyclists in urban traffic using improved YOLOv5 detector," *IET Image Processing*, vol. 15, pp. 3623–3637, 2021.
[9] "Real-time helmet violation detection in AI City Challenge," *arXiv preprint arXiv:2304.09248*, 2023.
[10] "Real-time multi-class helmet violation detection using YOLOv8 with license plate recognition," in *Springer Conf. Proc.*, 2024.
[11] "Helmet wear detection based on YOLOv5," in *Proc. 2nd Asia Conf. Algorithms, Computing and Machine Learning (ACM)*, 2023.
[12] "Helmet violation processing using deep learning," ResearchGate preprint, 2018.
[13] "Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8," *PMC*, 2024.
[14] "Research on the application of helmet detection based on YOLOv4," *SCIRP J.*, 2022.
[15] M. Dasgupta, O. Bandyopadhyay, and S. Chatterji, "Automated helmet detection for multiple motorcycle riders using CNN," in *2019 IEEE Conf. Information and Communication Technology (CICT)*, 2019.
[16] C. A. Rohith, S. A. Nair, P. S. Nair, S. Alphonsa, and N. P. John, "An efficient helmet detection for MVD using deep learning," in *2019 3rd Int. Conf. Trends in Electronics and Informatics (ICOEI)*, pp. 282–286, 2019.
[17] F. W. Siebert and H. Lin, "Detecting motorcycle helmet use with deep learning," *Accident Analysis & Prevention*, vol. 134, p. 105319, 2020.
[18] J. Terven, D. M. Córdova-Esparza, and J. A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, pp. 1680–1716, 2023.